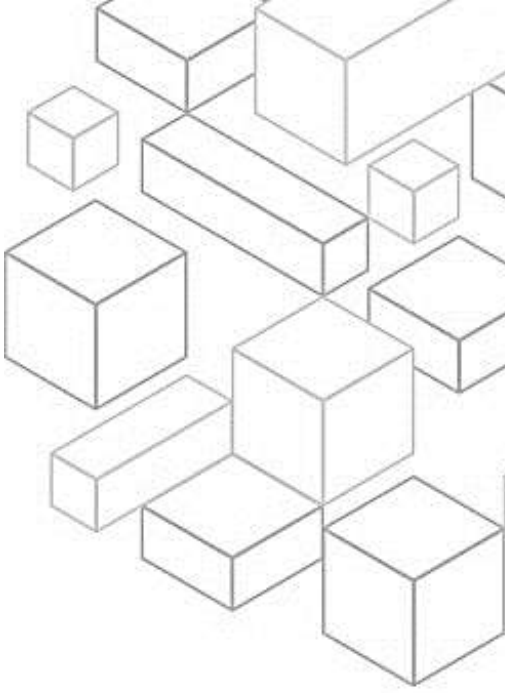


Module 7 Deep Dive on the Performance Efficiency Pillar

1.1 AWS Well-Architected

Welcome to module 7 of AWS Well-Architected: Deep Dive on the Performance Efficiency Pillar.

1.2 Learning objectives

In this module, you will get an overview of the performance efficiency pillar of the AWS Well-Architected Framework. You will also learn the design principles and best practices of the performance efficiency pillar.

1.3 Performance Efficiency Pillar Overview

To begin, you will get an overview of the performance efficiency pillar.

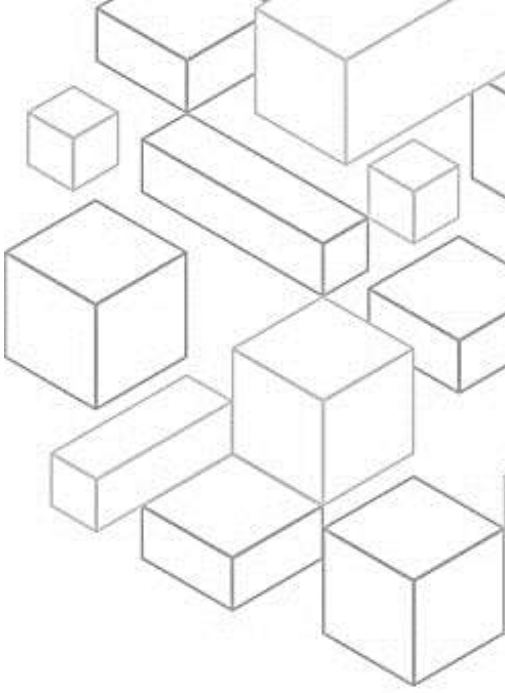
1.4 Pillars of Well-Architected

There are currently six pillars in the Well-Architected Framework: operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability. These pillars are the foundations of the architecture for your technology solutions in the cloud. This module will focus on the performance efficiency pillar.

1.5 What is the performance efficiency pillar?

What is the performance efficiency pillar? The performance efficiency pillar focuses on using computing resources efficiently to meet requirements and maintaining efficiency as demand changes and technologies evolve.

1.6 Performance Efficiency Design Principles



Now that you understand what the performance efficiency pillar is, you will dive deeper into the design principles of the performance efficiency pillar.

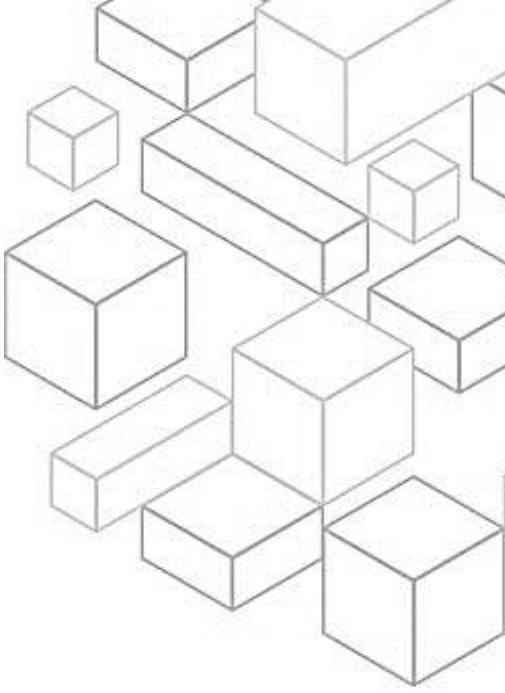
1.7 Performance Efficiency Design Principles

There are five design principles for performance efficiency in the cloud.

The first design principle is to democratize advanced technologies. Streamline advanced technology implementation for your team, and delegate complex tasks to your cloud vendor. Don't get your IT team to host a new technology. Instead, consider consuming it as a service. In the cloud, technologies that otherwise would require specialized expertise, such as machine learning and NoSQL databases, become services that your team can use. This gives them more time to focus on product development rather than spending time on provisioning and managing resources.

The second principle is to go global in minutes. Deploying into multiple Regions helps get your workload closer to your global audience. This can result in lower latency and a better experience. For example, by taking advantage of services such as AWS CloudFormation, you can quickly spin up resources in different global geographies with minimal overhead. This can offer better performance to application users and provide access to other features or functionality that may exist in different Regions.

Another principle is to use serverless architectures. Serverless architectures remove the need for you to run and maintain physical servers for traditional compute activities. For example, serverless storage services can act as static websites. This removes the need for web servers, and event services can host code. It also reduces the operational burden of managing physical servers. You



could benefit from lower transactional costs because managed services operate at cloud scale.

The fourth principle is to experiment more often. In the cloud, with virtually unlimited resources, you can quickly compare configurations on your workloads. This can be as simple as experimenting with a different sized instance or type of storage. Or it can be trying totally different services, such as running code in an AWS Lambda function rather than on an Amazon Elastic Compute Cloud, or Amazon EC2, instance.

The last design principle is to consider mechanical sympathy. Align your technology approach to your overall business goals, not the other way around. It is important to take time to understand how cloud services are consumed and use the technology approach that aligns best with your workload goals. For example, always consider data access patterns when you select database or storage approaches.

1.8 Performance Efficiency Best Practices

Now that you understand the performance efficiency principles, you will learn about the performance efficiency best practices.

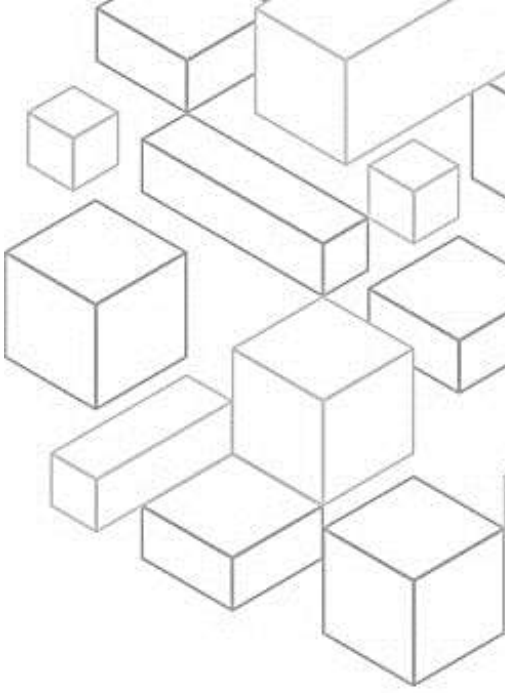
1.9 Performance Efficiency Best Practice Areas

The performance efficiency pillar is grouped into four areas of best practices. These include selection, review, monitoring, and trade-offs.

1.10 Selection

Selection is the first performance efficiency best practice area.

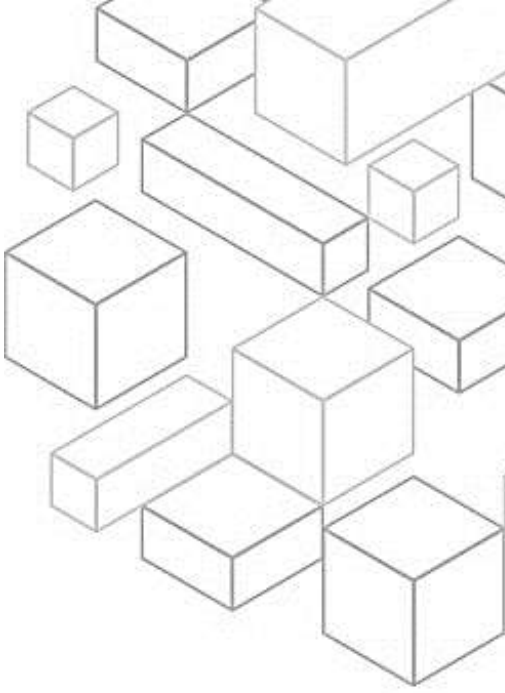
1.11 Performance architecture selection



For performance architecture selection, use a data-driven approach to select the patterns and implementation for your architecture and achieve a cost-effective solution. Your architecture will likely combine a number of architectural approaches. The implementation of your architecture will use services that are specific to the optimization of your architecture's performance. You should understand the available services and resources. Learn about the wide range of services and resources available in the cloud. You should also identify the relevant services and configuration options for your workload and understand how to achieve optimal performance.

You should define a process for architectural choices. To do this, use internal experience and knowledge of the cloud. You can also use external resources such as published use cases, documentation, or whitepapers to define a process to help choose resources and services. You should define a process that encourages experimentation and benchmarking with the services that could be used in your workload. Also, factor cost requirements into your decisions. Workloads often have cost requirements for operation. Use internal cost controls to select resource types and sizes based on predicted resource need. Determine which workload components could be replaced with fully managed services, such as managed databases, in-memory caches, and other services. By reducing your operational workload, you can focus resources on business outcomes.

You should use policies or reference architectures. Maximize performance and efficiency by evaluating internal policies and existing reference architectures. Then, use your analysis to select services and configurations for your workload. Use guidance from your cloud provider or an appropriate partner. Research



cloud company resources, such as solutions architects, professional services, or an appropriate partner, to guide your decisions. These resources can help review and improve your architecture for optimal performance.

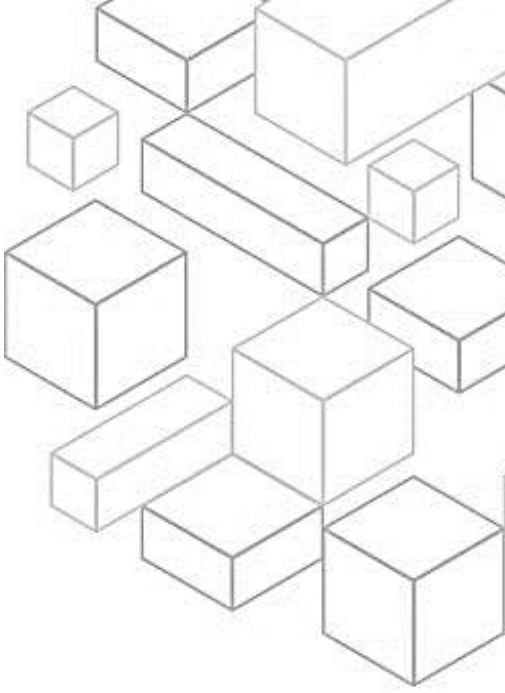
You should benchmark the performance of an existing workload to understand how it performs on the cloud. Use the data collected from benchmarks to drive architectural decisions. Use benchmarking with synthetic tests to generate data about how your workload's components perform. Benchmarking is generally quicker to set up than load testing and is used to evaluate the technology for a particular component.

Lastly, load test your workload. Deploy your latest workload architecture on the cloud using different resource types and sizes. Monitor the deployment to capture performance metrics that identify bottlenecks or excess capacity. Use this performance information to design or improve your architecture and resource selection.

1.12 Compute architecture selection

When selecting a compute architecture, choose compute resources that meet your requirements and performance needs while providing great efficiency of cost and effort. This can help you accomplish more with the same number of resources. But, the optimal compute solution for a workload will always vary based on your application design, usage patterns, and configuration settings. You should evaluate the available compute options. Understand the performance characteristics of the compute-related options available to you.

Know how instances, containers, and functions work, and what advantages or disadvantages they bring to your workload. In AWS, compute is available in three forms: instances, containers, and functions. Also, understand the

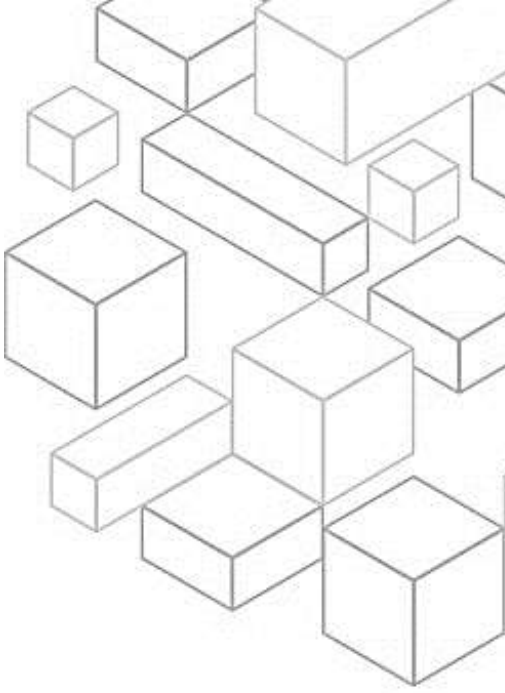


available compute configuration options. How do various options complement your workload, and which configuration options are best for your system? Examples include instance family, sizes, features such as GPU or I/O, function sizes, container instances, and single compared to multi-tenancy. You should also collect compute-related metrics. To understand how your compute resources are performing, you must record and track the utilization of various systems. You can use this data to more accurately determine resource requirements. Determine the required configuration by rightsizing.

Analyze performance characteristics of your workload and how these characteristics relate to memory, network, I/O, and CPU usage. Use this data to choose resources that best match your workload's profile. For example, a memory-intensive workload like a database might benefit from a higher memory per core ratio. However, a compute-intensive workload might need a higher core count and frequency, but can be satisfied with a lower amount of memory per core. You can also use the available elasticity of resources. The cloud provides the flexibility to expand and reduce your resources dynamically through a variety of mechanisms to meet changes in demand. Combining this elasticity with compute-related metrics, a workload can automatically respond to changes to use the resources it needs and only the resources it needs. Continually evaluate compute needs based on metrics. Use a data-driven approach to evaluate and optimize the compute resources for your workload over time.

1.13 Storage architecture selection

For storage architecture selection, the optimal storage solution for a system varies based on the kind of access method, which could be block, file, or object



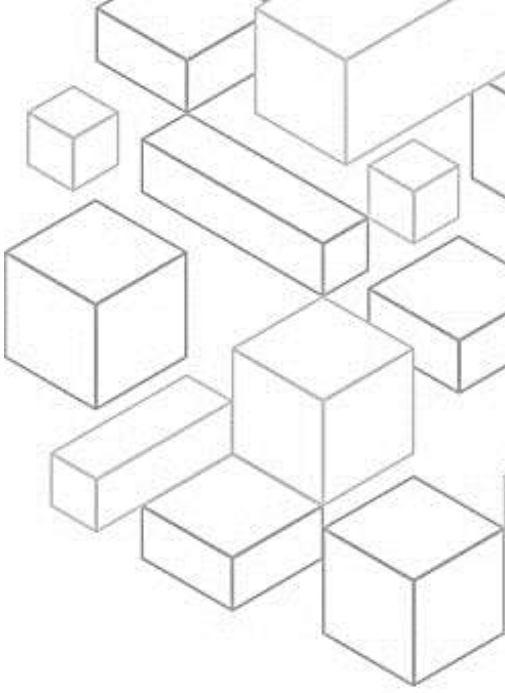
and based on if the patterns of access are random or sequential. The optimal storage solution varies based on required throughput and availability and durability constraints. It could also differ based on frequency of access or frequency of update. When selecting the storage solution for your architecture, you can follow a few best practices.

First, understand storage characteristics and requirements. Identify and document the workload storage needs, and define the storage characteristics of each location. Examples of storage characteristics include shareable access, file size, growth rate, throughput, input output operations per seconds or IOPS, latency, access patterns, and persistence of data. Use these characteristics to evaluate if block, file, object, or instance storage services are the most efficient solution for your storage needs. Next, evaluate available configuration options. Evaluate the various characteristics and configuration options and how they relate to storage. Understand where and how to use provisioned IOPS, solid state drive, or SSDs, magnetic storage, object storage, archival storage, or ephemeral storage to optimize storage space and performance for your workload.

The last best practice is to make decisions based on access patterns and metrics. Choose storage systems based on your workload's access patterns, and configure them by determining how the workload accesses data. Increase storage efficiency by choosing object storage over block storage. Configure the storage options you choose to match your data access patterns.

1.14 Database architecture selection

With regard to database architecture selection, choosing the wrong database solution and features for a system can lower performance efficiency. Therefore,

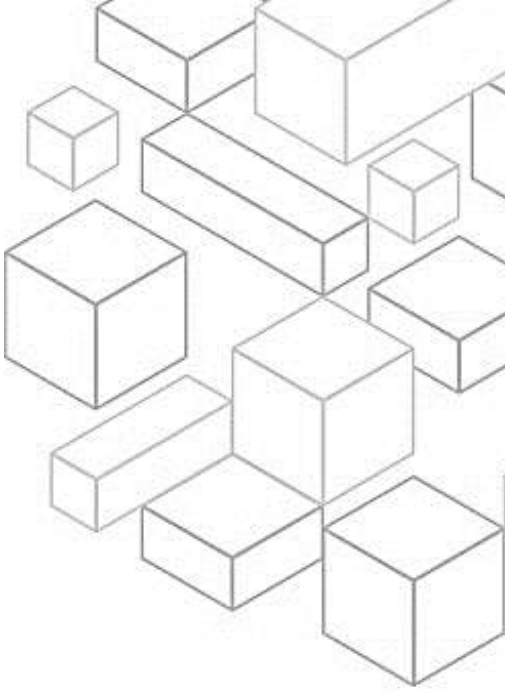


when deciding for an optimal database solution, it is important to consider the requirements. Examples include availability, consistency, partition tolerance, latency, durability, scalability, and query capability. You can follow best practices when selecting the optimal database solution for your workload. First, understand data characteristics.

Choose your data management solutions to optimally match the characteristics, access patterns, and requirements of your workload datasets. When selecting and implementing a data management solution, ensure that the querying, scaling, and storage characteristics support the workload data requirements. Learn how database options match your data models and which configuration options are best for your use case. Next, evaluate available options. Understand the available database options and how they can optimize your performance before you select your data management solution.

Use load testing to identify database metrics that matter for your workload. While you explore database options, consider options for parameter groups, storage, memory, compute, read replica, eventual consistency, connection pooling, and caching. Experiment with these configuration options to improve the metrics. Also, collect and record database performance metrics to understand how your data management systems are performing. These metrics will help you optimize your data management resources to ensure that your workload requirements are met and you have a clear overview on how the workload performs. Use tools, libraries, and systems that record performance measurements for database performance.

Choose data storage based on access patterns. Use the access patterns of the workload and requirements of the applications to decide on optimal data services and technologies to use. Lastly, optimize data storage based on access



patterns and metrics. Use performance characteristics and access patterns that optimize how data is stored or queried to achieve the best performance. Measure how optimizations such as indexing, key distribution, data warehouse design, or caching strategies impact system performance or overall efficiency.

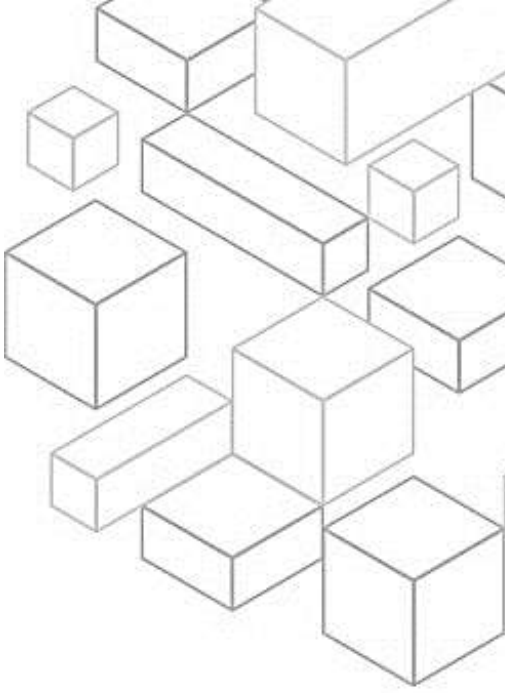
1.15 Network architecture selection

Network architecture selection can have great impacts (both positive and negative) on workload performance and behavior, because the network is between all workload components. There are also workloads that are heavily dependent on network performance such as high-performance computing, or HPC, in which deep network understanding is important to increase cluster performance. You must determine the workload requirements for bandwidth, latency, jitter, and throughput.

First, you should analyze and understand how network-related decisions impact workload performance. The network is responsible for connectivity between application components, cloud services, edge networks, and on-premises data. Therefore, it can highly impact workload performance. In addition to workload performance, user experience is also impacted by network latency, bandwidth, protocols, location, network congestion, jitter, throughput, and routing rules.

You should also evaluate networking features in the cloud that might increase performance. Measure the impact of these features through testing, metrics, and analysis. For example, take advantage of network-level features that are available to reduce latency, network distance, or jitter.

Choose appropriately sized dedicated connectivity or virtual private networks (VPNs) for hybrid workloads. When a common network is required to connect



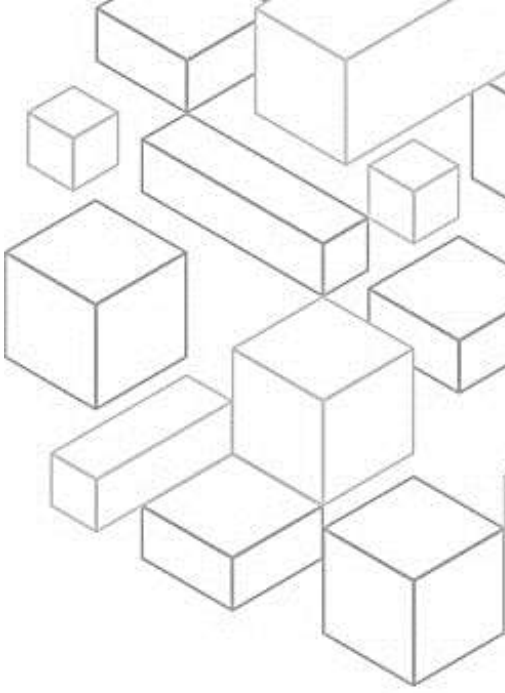
on-premises and cloud resources in AWS, verify that you have adequate bandwidth to meet performance requirements. Estimate the bandwidth and latency requirements for your hybrid workload. These numbers will drive the sizing requirements for your connectivity options.

Also, use load-balancing and encryption offloading. Load balancers can help achieve optimal performance efficiency of your target resources and improve system responsiveness.

You should assess the performance requirements for your workload, and choose the network protocols that optimize your workload's overall performance. There is a relationship between latency and bandwidth to achieve throughput. For instance, if your file transfer is using TCP, higher latencies will reduce overall throughput. There are approaches to fix this with TCP tuning and optimized transfer protocols. Some approaches use User Datagram Protocol, or UDP. The Scalable Reliable Datagram (SRD) protocol is a network transport protocol built by AWS for Elastic Fabric Adapters (EFAs) that provides reliable datagram delivery. Unlike the TCP protocol, SRD can reorder packets and deliver them out of order. This out-of-order delivery mechanism of SRD sends packets in parallel over alternate paths, increasing throughput.

You should also choose your workload's location based on network requirements. Evaluate options for resource placement to reduce network latency and improve throughput, providing an optimal user experience by reducing page load and data transfer times.

Lastly, you should optimize network configuration based on metrics. Improper network configuration can affect network performance, efficiency, and cost. In common network environments, to quickly complete early-stage deployment,



the proper network configuration is not fully considered in terms of network performance. To optimize your network configuration, you must first have visibility and data about your network environment. To understand how your network resources are performing, collect and analyze data to make informed decisions about optimizing your network configuration. Measure the impact of those changes and use the impact measurements to make future decisions.

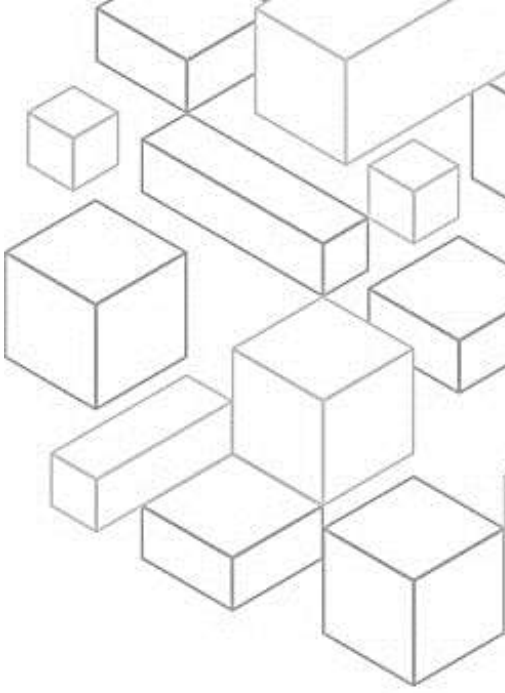
1.16 Review

Review is the second performance efficiency best practice area.

1.17 Evolve your workload to take advantage of new releases

Evolve your workload to take advantage of new releases when architecting workloads. There are finite options that you can choose from. However, over time, new technologies and approaches become available that could improve the performance of your workload. There are best practices you can follow to continually review and evolve your workloads to take advantage of new releases. First, evaluate ways to improve performance as new services, design patterns, and product offerings become available. Determine which of these could improve performance or increase the efficiency of the workload through evaluation, internal discussion, or external analysis.

Also, define a process to improve workload performance by evaluating new services, design patterns, resource types, and configurations as they become available. For example, run existing performance tests on new instance offerings to determine their potential to improve your workload. Lastly, evolve workload performance over time. As an organization, use the information



gathered through the evaluation process to actively drive adoption of new services or resources when they become available.

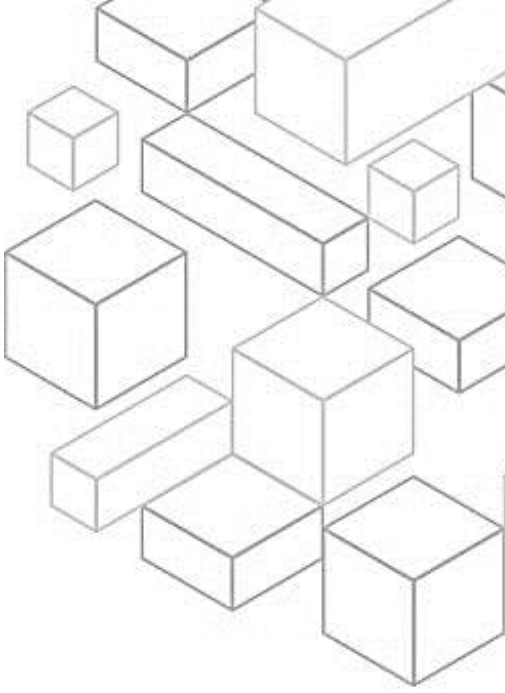
1.18 Monitoring

Monitoring is the third performance efficiency best practice area.

1.19 Monitor resources to ensure expected performance

Monitor resources to ensure expected performance after implementing your workload to remediate any issues or deviation from those expected performance levels. To do this, first use a monitoring and observability service to record performance-related metrics. For example, record database transactions, slow queries, I/O latency, HTTP request throughput, service latency, or other key data. You should also analyze metrics when events or incidents occur by using monitoring dashboards or reports to understand and diagnose the impact. These views provide insight into which portions of the workload are not performing as expected.

Establish key performance indicators, or KPIs, to measure workload performance. For example, an API-based workload might use overall response latency as an indication of overall performance. An ecommerce site might choose to use the number of purchases as its KPI. Then, with the performance-related KPIs that you defined, use a monitoring system that generates alarms automatically when these measurements are outside expected boundaries. You should also review metrics at regular intervals. As routine maintenance, or in response to events or incidents, review which metrics are collected. Use these reviews to identify which metrics were key in addressing issues and which



additional metrics, if they were being tracked, would help to identify, address, or prevent issues.

Lastly, monitor and alarm proactively. Use KPIs, combined with monitoring and alerting systems, to proactively address performance-related issues. Use alarms to automate actions and remediate issues where possible. Escalate the alarm to those able to respond if automated response is not possible. For example, you might have a system that can predict expected KPI values and send an alarm when they breach certain thresholds. You could also have a tool to automatically halt or roll back deployments if KPIs are outside of expected values.

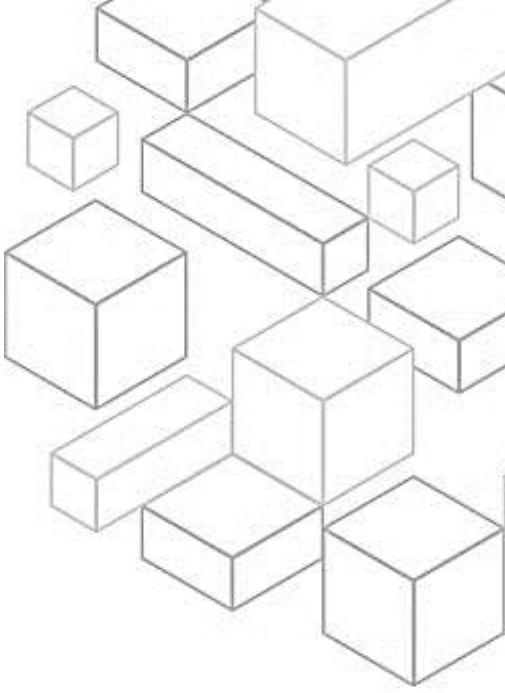
1.20 Trade-offs

Trade-offs is the final performance efficiency best practice area.

1.21 Using trade-offs to improve performance

Using trade-offs to improve performance when architecting solutions allows you to select an optimal approach. Often you can improve performance by trading consistency, durability, and space for time and latency. These best practices will help you answer the question of how to use trade-offs to improve performance. First, understand and identify areas where increasing the performance of your workload will positively impact efficiency or customer experience. For example, a website that has a large amount of customer interaction can benefit from using edge services to move content delivery closer to customers. Another best practice is to research and understand the various design patterns and services that help improve workload performance.

As part of the analysis, identify what you could trade to achieve higher performance. For example, using a cache service can help reduce the load



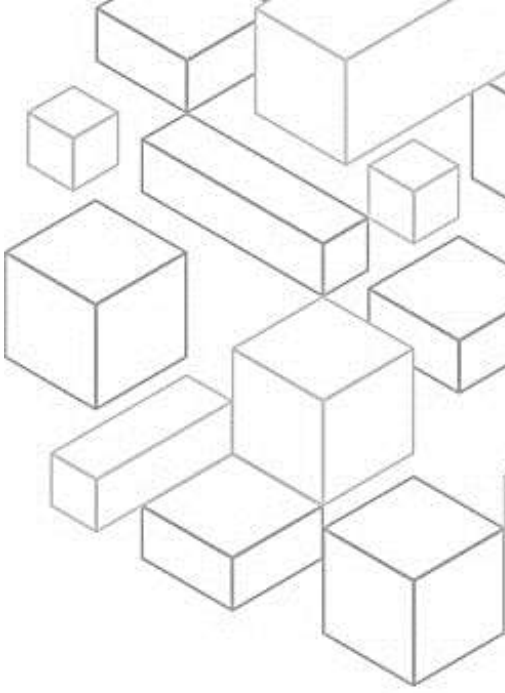
placed on database systems. However, this can require some engineering to implement safe caching or possible introduction of eventual consistency in some areas. Also, when evaluating performance-related improvements, determine which choices will impact your customers and workload efficiency. For example, if using a key-value data store increases system performance, it is important to evaluate how the eventually consistent nature of it will impact customers. Measure the impact of performance improvements.

As changes are made to improve performance, evaluate the collected metrics and data. Use this information to determine impact that the performance improvement had on the workload, the workload's components, and your customers. This measurement helps you understand improvements that result from the trade-off. It can also help you determine if any negative side effects were introduced. The last best practice is to use various performance-related strategies where applicable. For example, you can use strategies such as caching data to prevent excessive network or database calls.

You can use read replicas for database engines to improve read rates. You can try strategies such as sharding or compressing data where possible to reduce data volumes. You can also use buffering and streaming results as they are available to avoid blocking.

1.28 Summary

In this module, you learned about the performance efficiency pillar. This started with an overview and included in depth discussion on the value proposition, design principles, and best practices of the performance efficiency pillar.



1.29 Thank you

Thank you for your participation!